



《AI大模型Ragent项目》——三个子问题命中了八个意图，该保留哪几个

来自： 拿个offer-开源&项目实战

 马丁

2026年04月27日 22:27

开篇引言

上一篇讲了单次 LLM 调用怎么给叶子节点打分——Prompt 模板的两步判断流程、模型回复确定性、JSON 解析的五层容错。一次调用进去一个问题，出来一个按分数排好序的 List<NodeScore>，整个链路很清晰。

但实际场景不会这么简单。回忆一下第 4 篇查询改写——用户发了一句“我的订单到哪了，另外退货政策是什么？”，查询改写会把它拆成两个子问题，每个子问题独立走意图分类。如果用户的问题更复杂一些，拆出三个子问题也很正常。

这就带来了几个绕不开的问题：

- 三个子问题，要一个接一个串行调 LLM，还是同时并行调？
- 每个子问题最多返回 3 个意图，三个子问题就是最多 9 个，下游检索能承受吗？
- 如果简单地按分数从高到低截前 3 个，会不会出现某个子问题完全被忽略的情况？

用一个具体场景来感受这个问题。假设用户在电商客服界面发了一句长问题：

我的订单到哪了？另外之前买的 AirPods 能不能退，物流费怎么算？

查询改写把它拆成三个子问题。每个子问题各自命中了 2~3 个意图，加起来八个。系统总共只允许 3 个意图进入下游检索——该怎么选？这篇文章要回答的就是这个问题。IntentResolver 作为意图识别阶段的编排者，负责把子问题分发出去并行跑，把结果收回来做总量封顶，保证既不丢子问题，又不让意图数量爆炸。

并行调度：子问题怎么同时跑

1. resolve() 方法全景

IntentResolver 的入口方法 resolve() 做三件事：取子问题、并行分类、总量封顶。

```
@RagTraceNode(name = "intent-resolve", type = "INTENT")
public List<SubQuestionIntent> resolve(RewriteResult rewriteResult) {
    List<String> subQuestions = CollUtil.isEmpty(rewriteResult.subQuestions())
        ? rewriteResult.subQuestions()
        : List.of(rewriteResult.rewrittenQuestion());
    List<CompletableFuture<SubQuestionIntent>> tasks = subQuestions.stream()
        .map(q -> CompletableFuture.supplyAsync(
            () -> {
                try {
                    return new SubQuestionIntent(q, classifyIntents(q));
                } catch (Exception e) {
                    log.error("子问题意图分类失败，降级为空意图，question: {}", q, e);
                    return new SubQuestionIntent(q, List.of());
                }
            },
            intentClassifyExecutor
        ))
        .toList();
    List<SubQuestionIntent> subIntents = tasks.stream()
        .map(CompletableFuture::join)
        .toList();
    return capTotalIntents(subIntents);
}
```

逐段拆一下。

取子问题列表。 RewriteResult 来自第 4 篇查询改写，里面有 rewrittenQuestion（整体改写后的问题）和 subQuestions（拆分后的子问题列表）。如果有子问题列表就用子问题，没有（简单问题没拆）就把 rewrittenQuestion 当作唯一的子问题。一行代码，兜底逻辑到位。

并行提交。 每个子问题用 CompletableFuture.supplyAsync 提交到 intentClassifyExecutor 专用线程池。三个子问题三个 future，互不干扰。

等待汇总。`.join()` 等所有 `future` 完成，拿到每个子问题的意图分类结果，打包成 `List<SubQuestionIntent>`。
总量封顶。最后调 `capTotalIntents()` 做全局约束——总意图数不能超过 3 个。这是本篇的重头戏，后面单独展开。
在往下看之前，先认识一下两个数据模型：

```
// 子问题 + 它命中的意图列表
public record SubQuestionIntent(String subQuestion, List<NodeScore> nodeScores) {
}

// 封顶算法的中间结构：意图 + 它来自第几个子问题
public record IntentCandidate(int subQuestionIndex, NodeScore nodeScore) {
}
```

`SubQuestionIntent` 很直观——一个子问题文本，加上它命中的意图列表。`IntentCandidate`（意图候选）是封顶算法内部用的中间结构，给每个意图打上来自第几个子问题的标记，方便后面追踪归属。

2. 为什么一定要并行

2.1 串行的延迟账

假设每次意图分类的 LLM 调用耗时 800ms 左右（Prompt 几千 token，模型做相对复杂的分类判断）：

调度方式	三个子问题总延迟	说明
串行	~2400ms	三次调用依次执行，延迟累加
并行	~800ms	三个同时跑，取最慢那个

用户感知的首字延迟是整个 pipeline 的累加——会话记忆加载 + 查询改写 + 意图识别 + 检索 + LLM 生成。意图识别这一环如果串行，光这里就多出 1.6 秒，用户体验明显变差。并行跑只取最慢那个子问题的耗时，基本等于一次 LLM 调用的时间。

2.2 为什么用专用线程池

```
@Qualifier("intentClassifyThreadPoolExecutor")
private final Executor intentClassifyExecutor;
```

意图分类用的是专用线程池 `intentClassifyThreadPoolExecutor`，不和检索、记忆加载等异步任务混用。
为什么要隔离？意图分类是网络 IO 密集型任务（调用外部 LLM 服务），和检索（调用向量数据库）、记忆加载的 IO 特征不同。如果所有异步任务共用一个线程池，检索慢了（比如向量数据库偶尔抖动）可能把线程池占满，导致意图分类也卡住——明明子问题 LLM 调用已经返回了，拿不到线程执行后续逻辑。
线程池的核心参数：核心线程数等于 CPU 核心数，最大线程数是 CPU 核心数的两倍，队列用 `SynchronousQueue`（直接交接，不排队），拒绝策略是 `CallerRunsPolicy`（池子满了就让调用者线程自己跑）。这个配置思路是：子问题数量通常是 2~4 个，不需要很大的池子，但拒绝策略要兜底——哪怕池子满了也不能丢任务，降级到调用者线程同步执行就行。

3. 异常处理：lambda 里面的 try-catch

`resolve()` 里有一个容易被忽略的设计细节——`try-catch` 的位置。

```
.map(q -> CompletableFuture.supplyAsync(
    () -> {
        try {
            return new SubQuestionIntent(q, classifyIntents(q));
        } catch (Exception e) {
            log.error("子问题意图分类失败，降级为空意图, question: {}", q, e);
            return new SubQuestionIntent(q, List.of());
        }
    },
    intentClassifyExecutor
))
```

`try-catch` 包在 `lambda` 内部，而不是在 `.join()` 外面。这个位置的选择很有讲究。
如果 `try-catch` 放在外面（比如对 `.join()` 的结果做异常处理），单个子问题一旦抛异常，对应的 `CompletableFuture` 就会进入 `completeExceptionally` 状态。调 `.join()` 时会抛出 `CompletionException`，三个子问题里一个失败就可能中断整个流程。

放在里面就不一样了。不管内部怎么炸——网络超时、LLM 服务熔断、线程池拒绝——catch 里都会返回一个带空意图列表的 SubQuestionIntent(q, List.of())。CompletableFuture 始终正常完成，不会影响其他子问题的结果。

这里还有一个和第 6 篇的衔接点。classifyTargets() 内部已经有全局 try-catch 兜底（JSON 解析失败时返回空列表），这里再包一层是双重保险。classifyTargets() 兜的是 JSON 解析阶段的异常，resolve() 的 lambda 兜的是更外层的异常——比如 LLM 服务直接挂了、HTTP 连接超时、线程中断等，第一层根本走不到的场景。

4. .join() 的阻塞语义

```
List<SubQuestionIntent> subIntents = tasks.stream()
    .map(CompletableFuture::join)
    .toList();
```

.join() 会阻塞当前线程直到 future 完成。三个 future 虽然是挨个调 .join() 的，但它们在 intentClassifyExecutor 池子里已经并行跑了，所以总耗时不是三次 .join() 的累加，而是 max（三个 future 各自的耗时）。

打个比方：你同时点了三家外卖，然后坐在那里等。第一家 10 分钟到，第二家 12 分钟到，第三家 8 分钟到——你等到手的时间是 12 分钟，不是 30 分钟。.join() 的语义就是这样。

注意：resolve() 方法本身会阻塞当前线程（通常是 RAG pipeline 的主线程），但意图分类的 LLM 调用在 intentClassifyExecutor 池里异步执行，不占主线程资源。

单问题过滤：classifyIntents 的两道筛子

在子问题的结果进入总量封顶之前，每个子问题的意图列表先过一道本地筛子。

1. 分数过滤 + 数量限制

```
private List<NodeScore> classifyIntents(String question) {
    List<NodeScore> scores = intentClassifier.classifyTargets(question);
    return scores.stream()
        .filter(ns -> ns.getScore() >= INTENT_MIN_SCORE)
        .limit(MAX_INTENT_COUNT)
        .toList();
}
```

两个常量定义在 RAGConstant 类中，IntentResolver 通过静态导入引用：

INTENT_MIN_SCORE = 0.35：分数低于 0.35 的意图直接丢掉，不给下游添乱。

MAX_INTENT_COUNT = 3：单个子问题最多保留 3 个意图。

2. 和 Prompt 的微妙差异

Prompt 里告诉 LLM < 0.4 建议返回空数组，代码端的最低分数线是 0.35。为什么不一样？

Prompt 里 0.4 是给 LLM 的建议，LLM 不一定严格遵守。有些边界情况 LLM 可能给出 0.38 的分数，代码端用 0.35 做兜底，留一点容差空间。第 6 篇提到过这一点。

至于为什么不过滤得更严（比如直接用 0.6），是因为查询改写后的子问题可能稍微偏离原始问题的表述，导致分数比直接用原始问题打分时低一些。0.35 是一个有点相关的底线，保守一点留着让下游去判断，总比误杀强。

3. 三层筛子的全景

把 Prompt 层、单问题代码层、跨问题代码层放在一起看，意图经过了三层筛选：

阶段	筛选条件	作用
Prompt 层	评分标准 + 数量规则 + 歧义规则	约束 LLM 的输出行为
代码层（classifyIntents）	分数 >= 0.35 + 数量 <= 3	兜底 LLM 不遵守规则的情况
代码层（capTotalIntents）	总数 <= 3 + 多样性保证	跨子问题的全局约束

Prompt 管 LLM 的输出行为，代码管 LLM 不听话的情况，capTotalIntents 管多个子问题加在一起的总量。三层各管一件事，职责很清楚。

总量封顶：capTotalIntents 的三段式策略

这是本篇的核心内容。

1. 为什么需要总量封顶

还是用开头的场景。用户问“我的订单到哪了？另外之前买的 AirPods 能不能退，物流费怎么算？”，被拆成三个子问题：

子问题 A：我的订单到哪了？

子问题 B：AirPods 能不能退？

子问题 C：物流费怎么算？

三个子问题分别命中了这些意图：

子问题	命中意图	分数	类型
A	订单查询	0.92	MCP
A	订单详情	0.58	KB
B	退换货政策	0.88	KB
B	售后服务	0.62	KB
B	AirPods 产品介绍	0.45	KB
C	物流费规则	0.85	KB
C	跨境物流	0.52	KB
C	运费计算器	0.42	KB

一共 8 个意图。如果不做封顶，下游要跑 8 次操作（1 个 MCP 工具调用 + 7 个 KB 定向检索）：

检索延迟累加，首字响应飙到 3~4 秒

最终生成 Prompt 的上下文里塞入过多 chunks，核心信息被稀释

LLM 生成的 token 成本翻倍

所以必须有一个全局约束——不管你多少个子问题，最终进入下游的意图总数不能超过 MAX_INTENT_COUNT（3 个）。

2. 为什么不能简单截前 3 个

最直觉的做法：把 8 个意图按分数从高到低排，截前 3 个。

按照上面的场景，截前 3 个会是：

订单查询（A，0.92）

退换货政策（B，0.88）

物流费规则（C，0.85）

这个结果看起来不错，三个子问题各保留了一个。但换个场景就不行了。

如果子问题 A 是物流相关的长问题，刚好命中了三个高分意图：配送方式（0.92）、运费规则（0.90）、海外仓（0.88），子问题 B 关于退货的最高分退换货政策才 0.70。简单截前 3 个，A 独占三个名额，B 完全被忽略。结果就是用户问了两件事，系统只回答了物流，退货的问题完全没人管。

三种策略的对比：

策略	优点	缺点
全局分数截前 N	实现简单	部分子问题可能全军覆没
每子问题均摊	保证覆盖	低分意图挤掉高分意图
多样性 + 分数（Ragent 采用）	覆盖 + 质量平衡	实现稍复杂

Ragent 采用的是第三种：先保证每个子问题至少有一个代表，剩余名额再按分数全局竞争。

3. 算法五步详解

capTotalIntents() 的主方法结构很清晰，五个步骤依次推进：

```
private List<SubQuestionIntent> capTotalIntents(List<SubQuestionIntent> subIntents) {
    int totalIntents = subIntents.stream()
        .mapToInt(si -> si.nodeScores().size())
        .sum();

    // 未超限，直接返回
    if (totalIntents <= MAX_INTENT_COUNT) {
        return subIntents;
    }

    // 步骤1：收集所有意图，按子问题索引分组
    List<IntentCandidate> allCandidates = collectAllCandidates(subIntents);

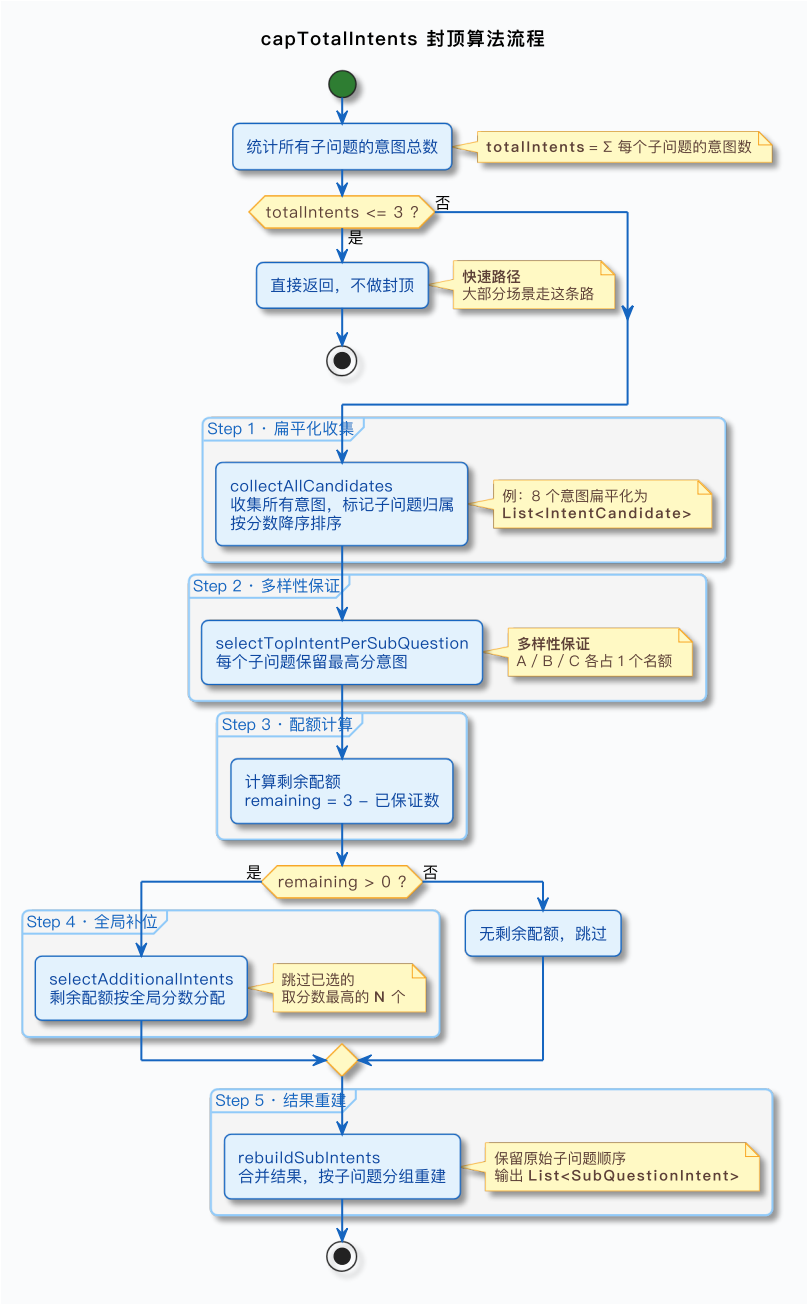
    // 步骤2：每个子问题保留最高分意图
    List<IntentCandidate> guaranteedIntents = selectTopIntentPerSubQuestion(allCandidates, subIntents);

    // 步骤3：计算剩余配额
    int remaining = MAX_INTENT_COUNT - guaranteedIntents.size();

    // 步骤4：从剩余候选中按分数选择
    List<IntentCandidate> additionalIntents = selectAdditionalIntents(allCandidates, guaranteedIntents, remaining);

    // 步骤5：合并并重建结果
    return rebuildSubIntents(subIntents, guaranteedIntents, additionalIntents);
}
```

用 PlantUML 画一下整体流程：



下面跟着八意图场景一步一步走。

3.1 快速放行

```
int totalIntents = subIntents.stream()
    .mapToInt(si -> si.nodeScores().size())
    .sum();

if (totalIntents <= MAX_INTENT_COUNT) {
    return subIntents;
}
```

先数一下总数。A 有 2 个，B 有 3 个，C 有 3 个，合计 8 个，超过 MAX_INTENT_COUNT (3)，不满足放行条件，继续往下走。
大部分场景其实都直接放行——用户问的是单问题（只有 1 个子问题，命中 1~2 个意图），或者简单复合问题（2 个子问题各命中 1 个，合计 2 个）。触发封顶算法的是少数情况，但这少数情况不处理好就会出问题。

3.2 步骤 1：收集所有候选 (collectAllCandidates)

```
private List<IntentCandidate> collectAllCandidates(List<SubQuestionIntent> subIntents) {
    List<IntentCandidate> candidates = new ArrayList<>();
    for (int i = 0; i < subIntents.size(); i++) {
        List<NodeScore> nodeScores = subIntents.get(i).nodeScores();
        if (CollUtil.isEmpty(nodeScores)) {
```

```
        continue;
    }
    for (NodeScore ns : nodeScores) {
        candidates.add(new IntentCandidate(i, ns));
    }
}
// 按分数降序排序
candidates.sort((a, b) -> Double.compare(b.nodeScore().getScore(), a.nodeScore().getScore()));
return candidates;
}
```

这一步做两件事：

1. 把三个子问题的意图扁平化成一个 `List<IntentCandidate>`，每个候选记录了这个意图来自第几个子问题（`subQuestionIndex`）。
2. 按分数降序排序。后续所有步骤都依赖这个全局顺序。

`IntentCandidate` 的作用在这里就体现出来了——它把意图和子问题索引绑定在一起。如果没有这个中间结构，后面做多样性保留的时候就不知道这个 0.88 的意图是 A 的还是 B 的。

用八意图场景来看，排序后的候选列表：

- ① 订单查询 (A, 0.92)
- ② 退换货政策 (B, 0.88)
- ③ 物流费规则 (C, 0.85)
- ④ 售后服务 (B, 0.62)
- ⑤ 订单详情 (A, 0.58)
- ⑥ 跨境物流 (C, 0.52)
- ⑦ AirPods 产品介绍 (B, 0.45)
- ⑧ 运费计算器 (C, 0.42)

3.3 步骤 2：每个子问题保留最高分（`selectTopIntentPerSubQuestion`）

这是多样性保证的核心。

```
private List<IntentCandidate> selectTopIntentPerSubQuestion(
    List<IntentCandidate> allCandidates, int subQuestionCount) {
    List<IntentCandidate> topIntents = new ArrayList<>();
    boolean[] selected = new boolean[subQuestionCount];

    for (IntentCandidate candidate : allCandidates) {
        int index = candidate.subQuestionIndex();
        if (!selected[index]) {
            topIntents.add(candidate);
            selected[index] = true;
        }
        // 所有子问题都有了保底意图，提前退出
        if (topIntents.size() == subQuestionCount) {
            break;
        }
    }
    return topIntents;
}
```

用 `boolean[] selected` 数组跟踪每个子问题是否已经拿到保底名额。按全局排好序的候选列表从高到低扫描，遇到未分配的子问题就给它一个名额。当所有子问题都至少有一个名额时，提前退出。

跟着八意图场景走一遍：

- 扫描到 ① 订单查询 (A, 0.92) → A 还没有 → 放入，`selected[A] = true`
- 扫描到 ② 退换货政策 (B, 0.88) → B 还没有 → 放入，`selected[B] = true`
- 扫描到 ③ 物流费规则 (C, 0.85) → C 还没有 → 放入，`selected[C] = true`
- 三个子问题都覆盖了，`topIntents.size() == 3 == subQuestionCount`，退出

guaranteedIntents = [订单查询 (A, 0.92), 退换货政策 (B, 0.88), 物流费规则 (C, 0.85)]

这一步的效果是：不管分数怎么分布，每个有意图的子问题至少保留一个代表。如果按前面那个极端场景——A 占了三个高分，这一步也会先给 B 留一个 0.70 的名额，而不是让 A 全部霸占。

特殊情况：某个子问题的 nodeScores 为空（LLM 什么都没匹配到），collectAllCandidates 里直接跳过了空列表，selected[index] 永远不会被设为 true。但循环不会卡住——要么所有有意图的子问题都覆盖了提前退出，要么候选列表扫完了自然退出。最终这个空子问题没有保底名额，但 rebuildSubIntents 会给它一个空的意图列表，结构上依然保留。

3.4 步骤 3：计算剩余配额

```
int remaining = MAX_INTENT_COUNT - guaranteedIntents.size();
```

上面三个子问题的场景，guaranteedIntents.size() = 3, remaining = 3 - 3 = 0。没有剩余配额了，步骤 4 直接跳过。
这恰好说明了一个关键特性：当子问题数量 >= MAX_INTENT_COUNT 时，每个子问题只能保留 1 个最高分意图，没有多余名额。三个子问题配三个名额，刚好一人一个。
但如果只有两个子问题呢？guaranteedIntents.size() = 2, remaining = 1，还能再塞一个。这就是步骤 4 的工作。

3.5 步骤 4：剩余配额按分数填 (selectAdditionalIntents)

```
private List<IntentCandidate> selectAdditionalIntents(
    List<IntentCandidate> allCandidates,
    List<IntentCandidate> guaranteedIntents,
    int remaining) {
    if (remaining <= 0) {
        return List.of();
    }

    List<IntentCandidate> additional = new ArrayList<>();
    for (IntentCandidate candidate : allCandidates) {
        // 跳过已经被选为保底的意图
        if (guaranteedIntents.contains(candidate)) {
            continue;
        }
        additional.add(candidate);
        if (additional.size() >= remaining) {
            break;
        }
    }
    return additional;
}
```

逻辑很直接：遍历全局排序的候选列表，跳过已经在 guaranteedIntents 里的，按分数从高到低取剩余配额个。
用另一个场景来展示这一步的效果。用户问我的订单到哪了？买的耳机能退吗？，拆成两个子问题：

- A（我的订单到哪了）命中：订单查询（0.90, MCP）、配送方式（0.85）、海外仓（0.60）
- B（买的耳机能退吗）命中：退换货政策（0.75）、售后维修（0.55）、商品信息（0.45）

步骤 2 选出保底：[订单查询 (A, 0.90), 退换货政策 (B, 0.75)]
剩余配额 = 3 - 2 = 1

步骤 4 从剩下的候选里按分数选：配送方式 (A, 0.85) 是剩余里最高的 → 选入
最终结果：A 留 2 个（订单查询 + 配送方式），B 留 1 个（退换货政策）。既保证了退货问题有代表，也让高分的物流意图多占一个名额。多样性和质量都兼顾了。

3.6 步骤 5：结构重建 (rebuildSubIntents)

```
private List<SubQuestionIntent> rebuildSubIntents(
    List<SubQuestionIntent> originalSubIntents,
    List<IntentCandidate> guaranteedIntents,
    List<IntentCandidate> additionalIntents) {
    // 合并所有选中的意图
    List<IntentCandidate> allSelected = new ArrayList<>(guaranteedIntents);
    allSelected.addAll(additionalIntents);
}
```



```
// 按子问题索引分组
Map<Integer, List<NodeScore>> groupedByIndex = new HashMap<>();
for (IntentCandidate candidate : allSelected) {
    groupedByIndex.computeIfAbsent(candidate.subQuestionIndex(), k -> new ArrayList<>())
        .add(candidate.nodeScore());
}

// 重建结果
List<SubQuestionIntent> result = new ArrayList<>();
for (int i = 0; i < originalSubIntents.size(); i++) {
    SubQuestionIntent original = originalSubIntents.get(i);
    List<NodeScore> retained = groupedByIndex.getOrDefault(i, List.of());
    result.add(new SubQuestionIntent(original.subQuestion(), retained));
}
return result;
}
```

这一步做的事情：把 `guaranteedIntents` 和 `additionalIntents` 合并，按 `subQuestionIndex` 分组，然后遍历原始子问题列表重建 `SubQuestionIntent`。

为什么要保留原始结构？因为下游（第 9 篇）要按子问题走定向检索，每个子问题的意图列表要明确关联到它的子问题文本。算法内部为了方便做全局排序，先把意图扁平化了；做完选择之后再按子问题分组塞回去，保证输出结构和下游约定一致。

注意 `getOrDefault(i, List.of())` 这个兜底——如果某个子问题的意图全部被淘汰了（或者它本来就没匹配到任何意图），重建的时候会给它一个空列表，不会丢失这个子问题的位置。

4. 完整演算：从 8 个到 3 个

把八意图场景的完整筛选过程走一遍，每一步看得清清楚楚。

输入：3 个子问题，8 个意图

A: 订单查询(0.92)，订单详情(0.58)

B: 退换货政策(0.88)，售后服务(0.62)，AirPods 产品介绍(0.45)

C: 物流费规则(0.85)，跨境物流(0.52)，运费计算器(0.42)

快速放行判断：8 > 3，不满足放行条件，触发封顶。

步骤 1 — 收集排序：

① 订单查询(A, 0.92) ② 退换货政策(B, 0.88) ③ 物流费规则(C, 0.85)

④ 售后服务(B, 0.62) ⑤ 订单详情(A, 0.58) ⑥ 跨境物流(C, 0.52)

⑦ AirPods 产品(B, 0.45) ⑧ 运费计算器(C, 0.42)

步骤 2 — 多样性保证：

扫描顺序	候选	子问题	已选?	动作
①	订单查询 0.92	A	否	✅ 选入，标记 A
②	退换货政策 0.88	B	否	✅ 选入，标记 B
③	物流费规则 0.85	C	否	✅ 选入，标记 C

`guaranteedIntents` = 3 个，A/B/C 各一个。

步骤 3 — 剩余配额：3 - 3 = 0，没有剩余。

步骤 4 — 跳过。

步骤 5 — 重建结果：

A: [订单查询(0.92)] ← 原来 2 个，保留 1 个

B: [退换货政策(0.88)] ← 原来 3 个，保留 1 个

C: [物流费规则(0.85)] ← 原来 3 个，保留 1 个

最终： 8 个意图 → 3 个意图。每个子问题都有代表，保留的都是各自最高分的。被淘汰的 5 个（订单详情、售后服务、AirPods 产品介绍、跨境物流、运费计算器）都是各子问题内的次要意图。

5. 设计权衡

5.1 为什么总量上限也是 3

单子问题的 `limit(MAX_INTENT_COUNT)` 和跨子问题的 `capTotalIntents` 用的是同一个常量 `MAX_INTENT_COUNT = 3`，但语义不同：

`classifyIntents()` 里的 `limit(3)` 是限制单个子问题的意图数——防止 LLM 返回太多
`capTotalIntents()` 里的 `<= 3` 是限制所有子问题合计的意图总数——控制下游检索的压力

为什么总数也是 3？下游检索阶段（第 9 篇详细讲）每个 KB 意图会走一次定向检索（从对应的 Milvus Collection 里按 topK 取 chunks），每个 MCP 意图走一次工具调用。意图太多会导致：

检索延迟累加——3 个定向检索已经需要并行优化了，更多就更难控制延迟
上下文稀释——检索回来的 chunks 都要塞进最终生成的 Prompt，太多 chunks 会稀释核心信息，LLM 回答质量反而下降
成本——更多 chunks 意味着更长的 Prompt，Token 消耗直接上升

3 是工程实践中的平衡点。再少（比如 1 个），复合问题就只能回答一部分；再多（比如 5 个），检索延迟和成本开始不可接受。

5.2 为什么多样性优先于分数

算法先做多样性保证（步骤 2），再做分数竞争（步骤 4）。为什么不反过来？
因为意图识别的目标是让用户每个子问题都有着落。如果先按分数全局竞争，再做多样性补救，可能已经来不及了——3 个名额全被分数最高的子问题占了，要补救就得踢掉已选的高分意图，逻辑复杂且结果不稳定。
先保证多样性，再用剩余名额做分数竞争，策略清晰，实现简单，结果也符合直觉——每个子问题至少有一个代表，剩下的名额给“最值得”的意图。

下游视角：封顶后的意图去哪了

封顶完成后，`resolve()` 返回 `List<SubQuestionIntent>`。下游还要做两件事，本篇简单提一下，第 8、9 篇详细展开。

1. mergeIntentGroup：按类型拆分

```
public IntentGroup mergeIntentGroup(List<SubQuestionIntent> subIntents) {
    List<NodeScore> mcpIntents = new ArrayList<>();
    List<NodeScore> kbIntents = new ArrayList<>();
    for (SubQuestionIntent si : subIntents) {
        mcpIntents.addAll(NodeScoreFilters.mcp(si.nodeScores()));
        kbIntents.addAll(NodeScoreFilters.kb(si.nodeScores()));
    }
    return new IntentGroup(mcpIntents, kbIntents);
}
```

封顶后的意图按类型拆成两组：MCP 意图和 KB 意图，装进 `IntentGroup` 里。MCP 意图走工具调用链路（比如订单查询要调用订单系统 API），KB 意图走定向检索链路（从对应的 Milvus Collection 里检索 chunks）。第 9 篇详细讲这两条链路怎么跑。
注意 `NodeScoreFilters` 是一个工具类，里面的 `mcp()` 方法过滤 MCP 类型且 `mcpToolId` 非空的意图，`kb()` 方法过滤 KB 类型的意图。封顶算法本身不关意图类型——它只看分数和子问题归属，类型拆分是下游的事。

2. isSystemOnly：系统类意图短路

```
public boolean isSystemOnly(List<NodeScore> nodeScores) {
    return nodeScores.size() == 1
        && nodeScores.get(0).getNode() != null
        && nodeScores.get(0).getNode().getKind() == SYSTEM;
}
```

如果只命中了一个意图，且类型是 `SYSTEM`（用户打招呼、说谢谢等社交性问题），就不用走检索和 LLM 生成了——直接短路返回预设回复，响应延迟只有几十毫秒。第 9 篇会讲具体的短路逻辑。

3. 歧义场景的特殊处理

封顶之后还有一个环节——歧义检测。如果封顶后的意图列表里出现了同名主题多分类的情况（比如用户问运费怎么算，运费规则和跨境运费计算都在名单里），系统会插入一轮引导，让用户选择想问的是国内运费还是跨境运费。

这个逻辑在 `capTotalIntents()` 之后、`mergeIntentGroup()` 之前触发。第 8 篇详细展开歧义引导的设计。

没有放之四海而皆准的数字

前面讲了 Ragent 的做法：`INTENT_MIN_SCORE = 0.35`、`MAX_INTENT_COUNT = 3`。这两个数字在电商客服场景下跑得不错，但换一个业务就不一定合适了。

举几个例子。如果你做的是法律咨询助手，用户问一个问题往往只对应一个法条或一个案由，意图命中非常集中，总量上限设成 2 甚至 1 就够了，多了反而引入噪音。如果你做的是医疗问诊分诊系统，用户描述的症状可能同时关联多个科室，3 个意图不够用，设成 5 个才能保证不漏诊。分数阈值也一样——知识库质量高、意图定义精准的系统，0.5 甚至 0.6 都没问题；知识库还在建设期、意图粒度比较粗的系统，0.35 甚至 0.3 才能保证召回率。

所以这两个参数没有通用的最优值，要根据你自己的业务场景做评估。建议的做法是：

- 1. 先用默认值跑起来，收集一批真实用户的查询日志
- 2. 统计每次请求命中的意图数量分布、分数分布，看看大多数请求落在什么范围
- 3. 挑出 bad case——哪些请求因为意图太多导致回答质量下降？哪些请求因为阈值太高导致有效意图被误杀？
- 4. 根据 bad case 调整参数，重新跑评估

核心思路就一个：参数服务于业务效果，而不是反过来。Ragent 选 3 和 0.35 是在通用场景下权衡了检索延迟、回答质量和成本之后的结果，而企业中的场景大概率需要你自己的数字。

小结

回顾本篇的核心要点：

- 1. 多个子问题通过 `CompletableFuture` + 专用线程池 `intentClassifyExecutor` 并行做意图分类，延迟从串行的 `sum` 降到并行的 `max`。
- 2. Lambda 内部的 `try-catch` 实现单问题失败降级，和 `classifyTargets()` 的全局 `try-catch` 形成双重兜底，不拖累其他子问题。
- 3. 单子问题先过两道筛子——分数过滤 (`>= 0.35`) + 数量限制 (`<= 3`)，兜底 LLM 不遵守 Prompt 规则的情况。
- 4. 总量封顶 `capTotalIntents` 实现跨子问题的全局约束——总数 `<= 3`，多样性优先于分数。
- 5. 算法五步走：统计总数 → 收集排序 → 多样性保证（每个子问题至少 1 个）→ 剩余配额按分数分配 → 重建结构。
- 6. `IntentCandidate` 作为中间结构，把意图和它所属的子问题索引绑定，让全局排序和子问题分组可以同时进行。
- 7. 封顶后的意图通过 `mergeIntentGroup` 按 KB/MCP 类型拆分，分别走检索和工具调用两条下游链路。

到这里，意图识别阶段的选的问题解决了——多少个意图该保留、怎么保证多样性、怎么控制总量。但还有一种情况封顶算法管不了：两个分类的意图同时命中了，分数都在 0.6 附近，系统不确定用户到底想问哪个。比如用户问“运费怎么算”，国内物流的运费规则和跨境物流的运费计算都命中了，分别给了 0.62 和 0.60。直接两个都查吗？还是应该反问用户？下一篇讲歧义引导——用户问“运费怎么算”，国内物流和跨境物流都举手了。